Introduction to Drones - 22AIE448

3D Mapping using a Monocular Camera and Control by Visual-Inertial Odometry (VIO)

Group CD2 : Ravishanmugam K, Ishwarya Murugesan, Aparna Bharani, Cibikumar B, Akhilan S

School of Artificial Intelligence, Amrita Vishwa Vidyapeetham

Toy model 3 - Model 2, Part 4 : state estimation with an Extended Kalman Filter (block 11)

Run M2_1 and M2_2 first (they create `dmdc_model.mat`). M2_3 is not needed but explains the controllers used here.

```
clearvars
clear all
ready = true;
PUBLISH = not(ready);   % make this PUBLISH = ready; to export the file as PDF
```

```
clearvars -except PUBLISH ready
```

Block 11 - Why Model 2 needs an estimator, and where it sits

In M2_3 the controllers were handed the **true state plus a little noise**: $u_k = -Kx_k$ for LQR and $x_0 = x_k$ for MPC. A real drone never sees x_k. Its sensors give only:

- the **monocular camera**: pixel positions of things on the floor (here: known markers, like AprilTags), 25 times per second;

- the **gyroscope**: the body rates ω.

Nothing measures the velocity v, and pixels are not positions. Block 11 turns these raw readings into the 12-number state $\hat{x}_k$ that blocks 9 and 10 need. It closes the loop of Model 2:

$$\text{apply to drone (10)} \rightarrow \text{drone moves} \rightarrow \text{camera + gyro} \rightarrow \text{EKF (11)} \rightarrow \hat{x}_k \rightarrow \text{MPC (9) / LQR} \rightarrow \text{apply (10)}$$

It keeps the idea of Model 2: the filter **predicts with the learned model** A, B (no mass, no inertia needed), the same model the controllers trust. It is **extended** because the camera equation is nonlinear (division by depth, rotation), so it must be linearized with a Jacobian at every step.

Toy example by hand - altitude from one pixel

A camera looks straight down from height h at a marker $d = 1$ m to the side. The pinhole camera puts it at pixel offset $u = fd/h$ - a curve, not a line. Its slope at our guess is $H = du/dh = -fd/h^2$. Prior: $\hat{h} = 2$ m, $\Sigma = 0.04$ m^2 (σ = 20 cm). Measured $u = 210$ px with noise variance $R_n = 4$ px^2:

$$u_{pred} = \frac{400 \cdot 1}{2} = 200, \quad H = -\frac{400}{4} = -100, \quad L = \frac{\Sigma H}{H^2\Sigma + R_n} = \frac{-4}{404} = -0.0099, \quad \hat{h} = 2 - 0.0099 \cdot 10 = 1.901$$

```
f = 400;  d = 1;            % focal length [px], marker 1 m to the side
```

```
h_hat = 2.0;  Sig_h = 0.04; % prior altitude [m] and its variance Sigma [m^2]
u_meas = 210;  R_u = 4;     % measured pixel offset [px] and its variance [px^2]
u_pred = f*d/h_hat
```

```
u_pred = 200
```

```
H = -f*d/h_hat^2            % slope of the pinhole curve at h_hat
```

```
H = -100
```

```
L = Sig_h*H/(H*Sig_h*H + R_u)   % estimator (Kalman) gain
```

```
L = -0.0099
```

```
h_new = h_hat + L*(u_meas - u_pred)
```

```
h_new = 1.9010
```

```
Sig_new = (1 - L*H)*Sig_h  % 0.04 -> 0.0004: sigma 20 cm -> 2 cm
```

```
Sig_new = 3.9604e-04
```

```
h_exact = f*d/u_meas        % the EKF differs from this only by the linearization
error
```

```
h_exact = 1.9048
```

The measurement model - camera pixels and gyroscope

The camera hangs under the drone looking down: camera axes in the body frame are x_c = forward, y_c = right, z_c = down, i.e. $R_{bc} = \mathrm{diag}(1,-1,-1)$. A marker at known position ℓ_j is seen from the drone at $R^T(\ell_j - p)$ (body frame), then in camera axes, then divided by its depth:

$$q_j = R_{bc}^T R(\theta)^T(\ell_j - p), \qquad z_j = \begin{bmatrix} f\ q_1/q_3 + c_x \\ f\ q_2/q_3 + c_y \end{bmatrix}, \qquad z_\omega = \omega \qquad \rightarrow (M2.21)$$

Jacobian. By the chain rule (pixel $\leftarrow q \leftarrow$ state). Moving the drone moves the marker the other way: $\partial q/\partial p = -R_{bc}^T R^T$. Turning the drone by a small $\delta\theta$: $R(\theta+\delta\theta) \approx R(\theta)\mathrm{Exp}(J_r\delta\theta)$, so $R^T a$ changes by $[R^T a]_\times J_r \delta\theta$. Velocity and rate do not move pixels (zero columns):

$$H_j = \frac{f}{q_3}\begin{bmatrix} 1 & 0 & -q_1/q_3 \\ 0 & 1 & -q_2/q_3 \end{bmatrix}\begin{bmatrix} -R_{bc}^T R^T & 0 & R_{bc}^T[R^T(\ell_j - p)]_\times J_r(\theta) & 0 \end{bmatrix} \qquad \rightarrow (M2.22)$$

$J_r(\theta) = I - \dfrac{1-\cos\phi}{\phi^2}[\theta]_\times + \dfrac{\phi - \sin\phi}{\phi^3}[\theta]_\times^2$, $\phi = \|\theta\|$ (the right Jacobian; $\approx I$ near hover). The gyroscope row is simply $[0 \quad 0 \quad 0 \quad I]$. Check against a numerical derivative:

```
load('dmdc_model.mat');           % A, B (learned), K (LQR), P_dare, Q, R (LQR
weights), dt, nSub, u_trim, P (drone)
C = DroneLib.camera();
x_test = [0.3; -0.2; 0.1; zeros(3,1); 0.05; -0.04; 0.2; zeros(3,1)];
[z, Hc, ids] = DroneLib.cam_measure(x_test, P.p_hover, C);
```

```
visible_markers = numel(ids)
```

```
visible_markers = 34
```

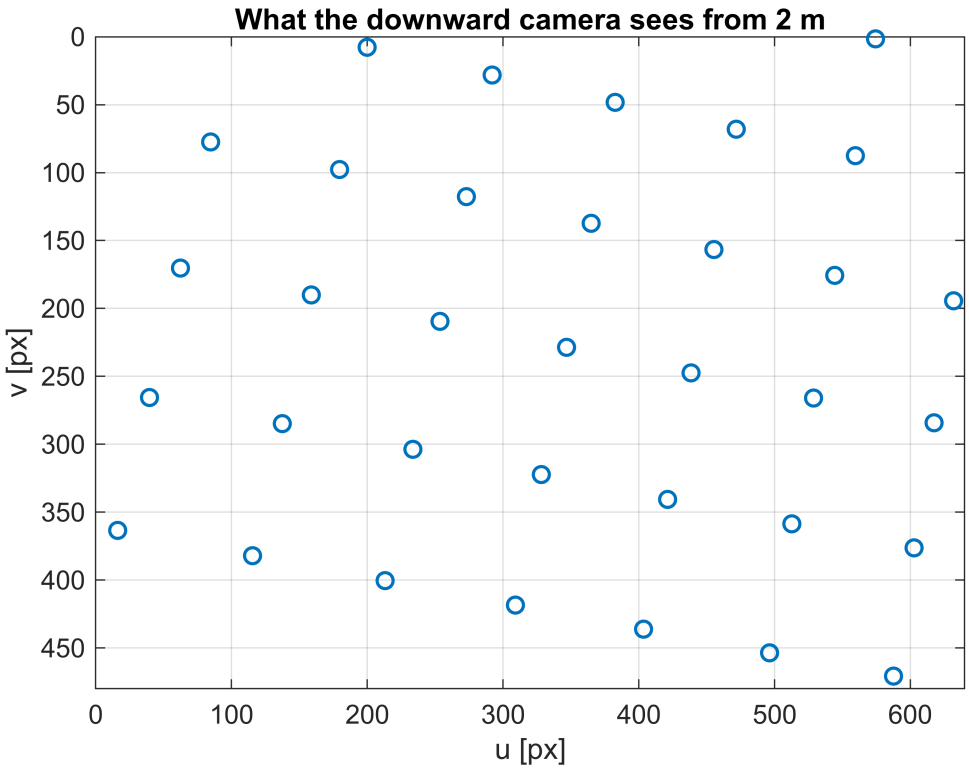
```
first_two_markers_px = reshape(z(1:4), 2, 2)
```

```
first_two_markers_px = 2×2
   16.1788   39.6013
  363.4334  265.5831
```

```
Hnum = zeros(size(Hc));  e = 1e-6;
for j = 1:12
    dx = zeros(12,1);  dx(j) = e;
    Hnum(:,j) = (DroneLib.cam_measure(x_test + dx, P.p_hover, C, ids) ...
                - DroneLib.cam_measure(x_test - dx, P.p_hover, C, ids))/(2*e);
end
max_jacobian_error = max(abs(Hc - Hnum), [], 'all')
```

```
max_jacobian_error = 1.1753e-07
```

```
figure
plot(z(1:2:end), z(2:2:end), 'o', LineWidth=1.2); set(gca, 'YDir', 'reverse')
axis equal; xlim([0 C.W]); ylim([0 C.Hpx]); grid on
xlabel("u [px]"); ylabel("v [px]"); title("What the downward camera sees from 2 m")
```



The prediction - with the learned model, and Q learned from data

The learned model is already linear, so the prediction needs no Jacobian ($F = A$):

$$\hat{x}_{k+1} = A\hat{x}_k + Bu_k, \qquad \Sigma_{k+1} = A\Sigma_k A^T + Q_e \qquad \rightarrow (M2.19)$$

How wrong is the learned model per step? The data tells us: on a **new** calibration flight (not used to learn A, B), collect what the model cannot explain:

$$Q_e = \mathrm{cov}(x_{k+1} - Ax_k - Bu_k) \qquad \rightarrow (M2.20)$$

The calibration flight had gusts of 0.1 N; the test flight below has 0.2 N (variance $\times 4$), so Q_e (variable `Q_ekf`) gets a safety factor of 4.

```
rng(5);
Kc = 625;  exc = zeros(4,1);
x = DroneLib.pack(P.p_hover + 0.3*randn(3,1), 0.1*randn(3,1),
DroneLib.expSO3(0.05*randn(3,1)), zeros(3,1));
Xc = zeros(12, Kc+1);  Uc = zeros(4, Kc);
for k = 1:Kc+1
    Xc(:,k) = DroneLib.to_dev(x, P.p_hover) + [1e-3*ones(6,1); 5e-4*ones(3,1);
1e-3*ones(3,1)].*randn(12,1);
    if k > Kc, break, end
    if mod(k-1, 5) == 0, exc = [2; 0.03; 0.03; 0.01].*(2*rand(4,1) - 1); end
    [~, ~, u_app] = DroneLib.mixer(u_trim + DroneLib.pilot(Xc(:,k), P) + exc, P);
    Uc(:,k) = u_app - u_trim;
    d = [0.1*randn(3,1); 2e-4*randn(3,1)];
    for j = 1:nSub, x = DroneLib.rk4(x, u_app, dt/nSub, P, d); end
end
E = Xc(:,2:end) - A*Xc(:,1:end-1) - B*Uc;   % what the learned model cannot explain
Q_ekf = 4*cov(E');
model_uncertainty_per_step = sqrt(diag(Q_ekf))'     % [m, m, m, m/s x3, rad x3,
rad/s x3]
```

```
model_uncertainty_per_step = 1×12
    0.0027    0.0028    0.0029    0.0062    0.0062    0.0061    0.0013    0.0015 ···
```

The correction

Letters. Model 2 already uses P (value matrix), Q (state weight) and K (LQR gain), so the filter writes Σ for its covariance, Q_e for its process noise and L for its gain. In the code they are `Pk`, `Q_ekf` and the gain inside `DroneLib.ekf2_update`.

Exactly the Kalman update of Model 1, but the 12-state of Model 2 is a plain vector (the attitude is already $\theta = \mathrm{Log}(R)$), so the correction is a plain addition:

$$L = \Sigma H^T \left(H\Sigma H^T + R_n\right)^{-1} \qquad \rightarrow (M2.23)$$

$$\hat{x} \leftarrow \hat{x} + L(z - h(\hat{x})), \qquad \Sigma \leftarrow (I - LH)\Sigma(I - LH)^T + LR_nL^T \qquad \rightarrow (M2.24)$$

The second form of Σ (Joseph form) equals $(I - LH)\Sigma$ when L is optimal, but stays symmetric and positive even with 70 measurements at once.

The closed loop - controllers flown on the EKF estimate

Same start as M2_3 (drone at $[1.5, -1, 0.8]$ m, target $[0, 0, 2]$ m), gusts, noisy pixels (1 px) and gyro (0.01 rad/s). The EKF starts **20 cm wrong**. Three runs: MPC on the EKF, LQR on the EKF, and MPC on the true state (as in M2_3) for comparison.

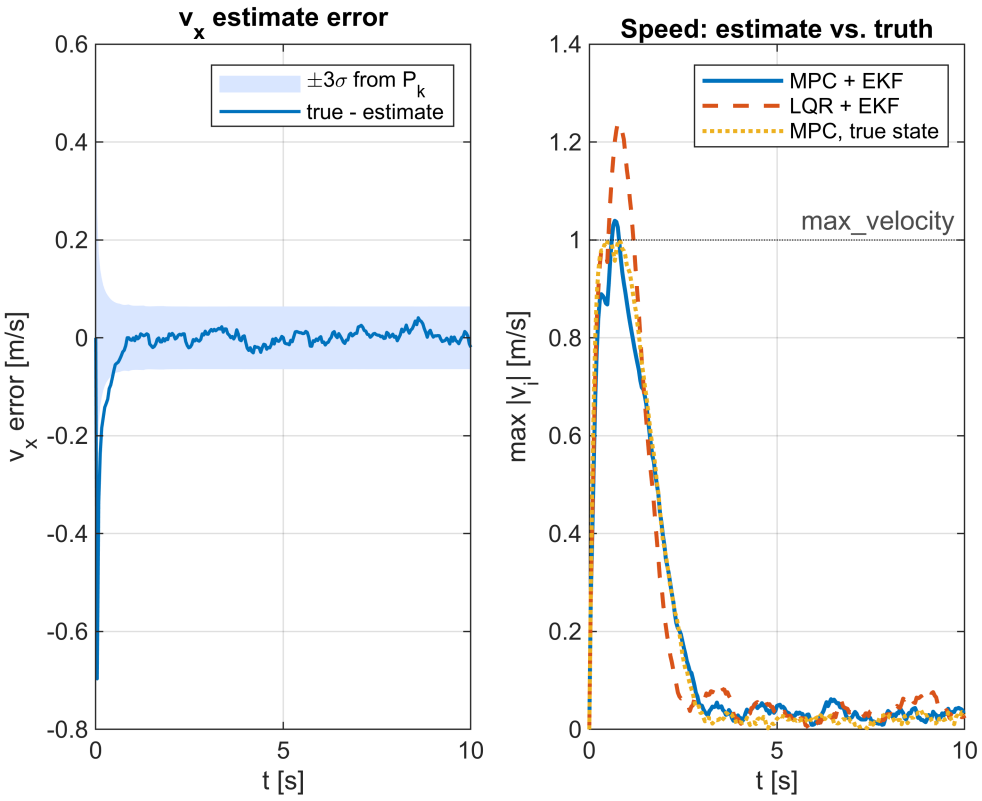
```
N = 25;
u_min = [5 - P.m*P.g0; -0.3; -0.3; -0.05];
u_max = [25 - P.m*P.g0; 0.3; 0.3; 0.05];
Cs = zeros(5,12);  Cs(1,4) = 1; Cs(2,5) = 1; Cs(3,6) = 1; Cs(4,7) = 1; Cs(5,8) = 1;
mpc = DroneLib.mpc_build(A, B, Q, R, P_dare, N, u_min, u_max, Cs, [1; 1; 1; 0.35;
0.35]);

rng(11);
Kn = 250;  t = (0:Kn)*dt;  names = ["MPC + EKF", "LQR + EKF", "MPC, true state"];
x0 = DroneLib.pack(P.p_hover + [1.5; -1.0; -1.2], zeros(3,1), eye(3), zeros(3,1));
for c = 1:3
    x  = x0;
    xh = DroneLib.to_dev(x, P.p_hover) + [0.2; -0.2; 0.1; zeros(3,1); 0.03; -0.03;
0.05; zeros(3,1)];
    Pk = blkdiag(0.04*eye(3), 0.04*eye(3), 0.0025*eye(3), 0.0025*eye(3));
    Xt = zeros(12, Kn+1);  Xh = Xt;  SD = Xt;
    for k = 1:Kn+1
        xd = DroneLib.to_dev(x, P.p_hover);
        [zc, ~, ids] = DroneLib.cam_measure(xd, P.p_hover, C);          % camera
sees the TRUE drone
        z = [zc + C.sigma_px*randn(size(zc)); xd(10:12) + C.sigma_gyro*randn(3,1)];
        [zh, Hc] = DroneLib.cam_measure(xh, P.p_hover, C, ids);         % ... the
EKF predicts from its estimate
        Hm = [Hc; zeros(3,9), eye(3)];
        Rn = blkdiag(C.sigma_px^2*eye(numel(zc)), C.sigma_gyro^2*eye(3));
        [xh, Pk] = DroneLib.ekf2_update(xh, Pk, z, [zh; xh(10:12)], Hm, Rn);
        Xt(:,k) = xd;  Xh(:,k) = xh;  SD(:,k) = sqrt(diag(Pk));
        if k > Kn, break, end
        if c == 3, xc = xd; else, xc = xh; end
        if c == 2, du = min(max(-K*xc, u_min), u_max); else, du =
DroneLib.mpc_solve(mpc, xc); end
        [~, ~, u_app] = DroneLib.mixer(u_trim + du, P);
        d = [0.2*randn(3,1); 5e-4*randn(3,1)];
        for j = 1:nSub, x = DroneLib.rk4(x, u_app, dt/nSub, P, d); end
        [xh, Pk] = DroneLib.ekf2_predict(xh, Pk, u_app - u_trim, A, B, Q_ekf);
    end
    err = Xt - Xh;  late = t > 1;
    fprintf("%s\n   estimate error: position %.1f mm, velocity %.1f cm/s (never
measured)\n" + ...
            "   flight: max |v| = %.3f m/s, distance after 10 s = %.3f m\n",
names(c), ...
            1000*sqrt(mean(sum(err(1:3,late).^2))),
100*sqrt(mean(sum(err(4:6,late).^2))), ...
            max(abs(Xt(4:6,:)), [], 'all'), norm(Xt(1:3,end)));
```

```
    if c == 1, Err1 = err;  SD1 = SD;  end
    Vmax(c,:) = max(abs(Xt(4:6,:)));
end
```

```
MPC + EKF
   estimate error: position 3.4 mm, velocity 2.8 cm/s (never measured)
   flight: max |v| = 1.039 m/s, distance after 10 s = 0.029 m
LQR + EKF
   estimate error: position 3.5 mm, velocity 3.8 cm/s (never measured)
   flight: max |v| = 1.239 m/s, distance after 10 s = 0.059 m
MPC, true state
   estimate error: position 3.5 mm, velocity 2.6 cm/s (never measured)
   flight: max |v| = 1.000 m/s, distance after 10 s = 0.029 m
```

```
figure
subplot(1,2,1)
fill([t fliplr(t)], [3*SD1(4,:) -fliplr(3*SD1(4,:))], [0.85 0.9 1],
EdgeColor="none"); hold on
plot(t, Err1(4,:), LineWidth=1.2); hold off; grid on
xlabel("t [s]"); ylabel("v_x error [m/s]"); legend("\pm3\sigma from P_k", "true -
estimate")
title("v_x estimate error")
subplot(1,2,2)
plot(t, Vmax(1,:), t, Vmax(2,:), '--', t, Vmax(3,:), ':', LineWidth=1.5); hold on
yline(1, 'k:', "max\_velocity"); hold off; grid on
xlabel("t [s]"); ylabel("max |v_i| [m/s]"); legend(names)
title("Speed: estimate vs. truth")
```



What to notice. The position is known to a few millimetres, the velocity to about 3 cm/s although it is never measured, and the errors stay inside the $\pm 3\sigma$ band that P_k predicts, so the filter knows how uncertain it is. The first ~0.5 s show the filter converging from its 20 cm wrong start. MPC on the estimate can overshoot the 1 m/s limit by a few percent, because it enforces the limit on the **estimated** state; the standard fix is a small margin (e.g. plan with 0.95 m/s).

In the real drone: the camera pixels come from the monocular camera stream, the gyro from the IMU topic, and u_k is the command we send. ArduPilot's own EKF3 already publishes `/ap/v1/pose/filtered`; block 11 is what lets Model 2 run on our own VIO measurements instead.

```
if(PUBLISH == ready)
    mlxfile = matlab.desktop.editor.getActive().Filename;
    [folder, name, ext] = fileparts(mlxfile);
    outfilepdf = fullfile(folder, 'pdf', [name, ext, '.pdf']);
    export(mlxfile, outfilepdf);
    if ispc
        winopen(outfilepdf);
    elseif ismac
        system(['open ' char(outfilepdf)]);
    else
        system("env -u LD_LIBRARY_PATH xdg-open '" + outfilepdf + "' &");
    end
end
```